

# Tarsau

## Dosya Arşivleme Aracı

### Proje Geliştirme Raporu

2. Öğretim A Grubu

Ömer Faruk TÜRKDOĞDU – G231210002

Mehmet Kaan KURT – G231210008

---

#### Proje Özeti

C programlama dili ile geliştirilmiş, tar/rar/zip'e benzer şekilde ASCII metin dosyalarını .sau formatında arşivleyen ve açan bir komut satırı aracı.

Geliştirme Tarihi: Mayıs 2026  
Dil: C (POSIX.1-2008)  
GitHub: <https://github.com/tarsau-project/tarsau>

# 1. Giriş

tarsau, C programlama dili ile geliştirilmiş, dosya arşivleme ve açma işlemlerini gerçekleştiren bir komut satırı aracıdır. Proje; **tar**, **rar**, **zip** gibi yaygın arşivleme programlarından ilham alarak tasarlanmış olmakla birlikte sıkıştırma işlemi yapmayı hedeflememekte, yalnızca birden fazla ASCII metin dosyasını tek bir **.sau** uzantılı arşiv dosyasında birleştirmeyi amaçlamaktadır.

Proje, sistem programlama temel kavramlarını (dosya G/C, bellek yönetimi, POSIX sistem çağrıları) pratik bir uygulama üzerinden pekiştirmek amacıyla tasarlanmıştır. Geliştirme sürecinde kod yazımı, hata yönetimi, test ve dokümantasyon aşamaları titizlikle takip edilmiştir.

## 1.1 Amaç ve Kapsam

- ASCII metin dosyalarını tek bir arşiv altında toplamak
- Arşivden orijinal dosyaları geri yükleyebilmek
- Dosya izinlerini (.sau formatı içerisinde) korumak
- Maksimum 32 dosya ve 200 MB toplam boyut kısıtlamalarını uygulamak
- Hatalı veya bozuk arşiv dosyalarını güvenli şekilde algılamak

## 1.2 Kullanılan Teknolojiler

| Teknoloji / Araç | Açıklama                                                     |
|------------------|--------------------------------------------------------------|
| C (POSIX.1-2008) | Ana geliştirme dili; sistem çağrıları için POSIX uzantıları  |
| GCC              | Derleme; -Wall -Wextra -pedantic bayraklarıyla hata denetimi |
| GNU Make         | Derleme otomasyonu (Makefile)                                |
| Git & GitHub     | Surum kontrolü ve uzak depo yönetimi                         |
| Valgrind         | Bellek sızıntısı analizi                                     |
| Linux (Ubuntu)   | Geliştirme ve test ortamı                                    |

## 2. Geliştirme Sureci

Proje, artımlı (incremental) bir yaklaşımla geliştirilmiştir. Her asama, çalışan ve test edilmiş bir bileşenin üretilmesiyle tamamlanmıştır. Git commit geçmişi bu sureci doğrusal şekilde yansıtmaktadır.

### 2.1 Asama 1: Proje Yapılanması ve Temel Giriş Doğrulama

İlk aşamada projenin dizin yapısı oluşturulmuş, `main()` fonksiyonu iskelet olarak yazılmış ve komut satiri argümanlarının ayrıştırılması (`-b` / `-a` modları) hayata geçirilmiştir.

#### Yapılan işler:

- `src/tarsau.c` dosyasının oluşturulması
- `argc/argv` doğrulaması ve kullanım mesajının yazılması
- Makefile hazırlanması (derleme, temizlik hedefleri)
- GitHub deposunun oluşturulması ve ilk commit

#### Commit mesajı:

```
feat: proje iskeleti oluşturuldu, temel CLI ayrışması eklendi
```

### 2.2 Asama 2: `metin_mi()` Fonksiyonu ve ASCII Doğrulama

Yalnızca ASCII metin dosyalarının arşivlenmesi gerektiği için, `metin_mi()` fonksiyonu geliştirilmiştir. Bu fonksiyon bir dosyayı bayt bayt okuyarak ikili (binary) veri içerip içermediğini denetler.

#### Kod parçacığı: `metin_mi()` fonksiyonu

```
int metin_mi(const char *yol) {
    FILE *fp = fopen(yol, "rb");
    if (!fp)
        return 0;

    int c;
    while ((c = fgetc(fp)) != EOF) {
        if (c > 127) {
            fclose(fp);
            return 0;
        }
        if (c < 32 && c != '\t' && c != '\n' && c != '\r') {
            fclose(fp);
            return 0;
        }
    }
    fclose(fp);
    return 1;
}
```

Fonksiyon; sekme (`\t`), yeni satır (`\n`) ve satir başı (`\r`) karakterlerini metin olarak kabul eder; 127 üstü baytları ve diğer kontrol karakterlerini ise ikili veri olarak değerlendirir.

### 2.3 Asama 3: `.sau` Arşiv Formatı Tasarımı ve Oluşturma

Arşiv dosyasının formatının tasarlanması, projenin en kritik adımlarından birini oluşturmaktadır. **.sau** formatı iki ana bölümden oluşur:

- Organizasyon (header) bölümü: dosya metadata'sini içerir
- İçerik bölümü: dosyaların ham verisini sırasıyla barındırır

#### Arşiv Formatı Yapısı:

```
[10 bayt uzunluk][|dosya1,izin,boyut|dosya2,izin,boyut|...][icerik1][icerik2]...
```

#### Örnek: t1.txt (100 bayt, 644) ve t2.txt (50 bayt, 755)

```
0000000035|t1.txt,644,100||t2.txt,755,50|<t1 icerigi><t2 icerigi>
```

#### Kod parçacığı: **arsiv\_olustur()** fonksiyonu (organizasyon yazımı)

```
char org[MAX_DOSYA * 300] = "";

for (int i = 0; i < dosya_sayisi; i++) {
    char kayit[512];
    sprintf(kayit, "|%s,%o,%ld|",
            bilgiler[i].isim,
            bilgiler[i].izin & 0777,
            bilgiler[i].boyut);
    strcat(org, kayit);
}

long org_uzunluk = strlen(org);
char baslik[11];
sprintf(baslik, "%010ld", org_uzunluk);

fwrite(baslik, 1, 10, cikti);
fwrite(org, 1, org_uzunluk, cikti);
```

## 2.4 Asama 4: Organizasyon Parse Etme ve Arşiv Açma

Arşiv açma işlemi **arsiv\_ac()** ve **kayitlari\_parse\_et()** fonksiyonlarıyla gerçekleştirilmiştir. Parse işlemi, organizasyon bölümündeki pipe-delimited (|) kayıtları dosya adına, izinlerine ve boyutuna ayırır.

#### Kod parçacığı: **kayitlari\_parse\_et()** - temel ayırma döngüsü

```
while (*p) {
    if (*p != '|') { p++; continue; }
    p++;

    char *son = strchr(p, '|');
    if (!son) break;
    *son = '\0';

    char *isim = strtok(p, ",");
    char *izin_str = strtok(NULL, ",");
    char *boyut_str = strtok(NULL, ",");

    strncpy(bilgiler[*dosya_sayisi].isim, isim, 255);
    bilgiler[*dosya_sayisi].izin = strtol(izin_str, NULL, 8);
    bilgiler[*dosya_sayisi].boyut = atol(boyut_str);
```

```

    (*dosya_sayisi)++;
    p = son + 1;
}

```

## 2.5 Asama 5: Hata Yönetimi ve Dayanıklılık

Programın güvenli çalışabilmesi için kapsamlı hata denetim mekanizmaları eklenmiştir. Aşağıdaki tablo kontrol edilen durumları özetlemektedir:

| Hata Durumu          | Kontrol Yeri               | Kullanıcı Mesajı               |
|----------------------|----------------------------|--------------------------------|
| Dosya bulunamadı     | stat() doğrulama           | "dosya.txt: Dosya bulunamadı"  |
| İkili (binary) dosya | metin_mi()                 | "format uyumsuzluğu!"          |
| 32'den fazla dosya   | argc sayacı                | "En fazla 32 dosya"            |
| 200 MB asimi         | toplam boyut kontrolü      | "200MB siniri aşıyor"          |
| Bozuk .sau arşivi    | baslik + boyut doğrulama   | "uygunsuz veya bozuk!"         |
| Uzun dosya adı       | strlen > 255 kontrolü      | "Dosya adı çok uzun"           |
| Tekrarlanan dosya    | çift döngü karşılaştırması | "birden fazla kez belirtilmiş" |

### Arşiv doğrulama kontrolü (arsiv\_ac içinden):

```

/* Baslik rakam doğrulama */
for (int i = 0; i < 10; i++) {
    if (baslik[i] < '0' || baslik[i] > '9') {
        printf("Arşiv dosyası uygunsuz veya bozuk!\n");
        fclose(fp);
        return 0;
    }
}

/* Dosya boyutu tutarlılık kontrolü */
if (fstat(fileno(fp), &arsiv_st) != 0 ||
    arsiv_st.st_size < 10 + org_uzunluk) {
    printf("Arşiv dosyası uygunsuz veya bozuk!\n");
    fclose(fp);
    return 0;
}

```

## 3. Kod Yapılanması

### 3.1 Fonksiyon Listesi

| Fonksiyon            | İmza                                              | Açıklama                                      |
|----------------------|---------------------------------------------------|-----------------------------------------------|
| metin_mi()           | int metin_mi(const char *yol)                     | Dosyanın ASCII metin olup olmadığını denetler |
| arsiv_olustur()      | int arsip_olustur(bilgi*, int, const char*)       | .sau arşivini oluşturur ve dosyaları yazar    |
| kayitlari_parse_et() | int kayitlari_parse_et(const char*, bilgi*, int*) | Organizasyon stringini ayrıştırır             |
| arsiv_ac()           | int arsip_ac(const char*, const char*)            | Arşivi hedef dizine acar                      |
| main()               | int main(int argc, char *argv[])                  | Giriş noktası; CLI ayrışması ve işlemleri     |

### 3.2 Veri Yapıları

Program, dosya metadatasını taşımak için aşağıdaki yapıyı kullanır:

```
struct dosya_bilgi {
    char isim[256]; /* Temel dosya adi (yol içermez) */
    char yol[256]; /* Tam veya göreli kaynak yolu */
    mode_t izin; /* POSIX dosya izin bitleri */
    long boyut; /* Bayt cinsinden dosya boyutu */
};
```

#### Kısıtlamalar:

- [object Object] - izin verilen maksimum dosya sayısı
- [object Object] - toplam içerik boyutu üst sınırı
- [object Object] - G/C tampon boyutu

## 4. Örnek Kullanım Senaryoları ve Ekran Çıktıları

### 4.1 Derleme

```
$ make
gcc -Wall -Wextra -pedantic -std=c11 -o tarsau src/tarsau.c
$
```

### 4.2 Basit Arşiv Oluşturma

Uç metin dosyasını varsayılan **a.sau** arşiviyle birleştirme:

```
$ echo 'Merhaba Dünya!' > selam.txt
$ echo 'Benim ikinci dosyam.' > notlar.txt
$ printf 'Ali\nVeli\nAyse' > isimler.txt

$ ./tarsau -b selam.txt notlar.txt isimler.txt
Dosyalar başarıyla birleştirildi.

$ ls -lh a.sau
-rw-r--r-- 1 user user 110 May 12 14:23 a.sau
```

### 4.3 Özel Çıktı Adıyla Arşiv Oluşturma

```
$ ./tarsau -b selam.txt notlar.txt isimler.txt -o proje.sau
Dosyalar başarıyla birleştirildi.

$ ls -lh proje.sau
-rw-r--r-- 1 user user 110 May 12 14:25 proje.sau
```

### 4.4 Arşivi Geçerli Dizine Açma

```
$ ./tarsau -a proje.sau
selam.txt, notlar.txt, isimler.txt dosyaları acildi.

$ cat selam.txt
Merhaba Dünya!
```

### 4.5 Arşivi Belirli Bir Dizine Açma

```
$ ./tarsau -a proje.sau çıktı_klasörü
cikti_klasoru dizininde selam.txt, notlar.txt, isimler.txt dosyaları acildi.

$ ls cikti_klasoru/
isimler.txt  notlar.txt  selam.txt
```

### 4.6 Hata Senaryoları

**Senaryo A: Binary dosya arşivlemeye çalışma**

```
$ ./tarsau -b /usr/bin/ls metin.txt
/usr/bin/ls giriş dosyasının formatı uyumsuz!
```

### Senaryo B: 200 MB sınırı asımı

```
$ ./tarsau -b buyuk1.txt buyuk2.txt  
Hata: Toplam dosya boyutu 200MB sınırını aşıyor.
```

### Senaryo C: Bozuk .sau dosyası açma girişimi

```
$ ./tarsau -a bozuk_dosya.sau  
Arşiv dosyası uygunsuz veya bozuk!
```

### Senaryo D: Yanlış uzantılı dosya

```
$ ./tarsau -a arxiv.tar  
Arşiv dosyası uygunsuz veya bozuk!
```

### Senaryo E: Tekrarlanan dosya belirtme

```
$ ./tarsau -b metin.txt metin.txt  
Hata: Aynı dosya birden fazla kez belirtilmiş: metin.txt
```

### Senaryo F: Kullanım yardımı (argümentsiz çalıştırma)

```
$ ./tarsau  
Kullanım:  
tarsau -b dosya1 dosya2 ... [-o arxiv.sau]  
tarsau -a arxiv.sau [hedef_dizin]
```



## 5. Git Surum Kontrolü ve GitHub Entegrasyonu

Proje, başından itibaren surum kontrolü altında tutulmuştur. Her önemli geliştirme aşaması ayrı bir commit ile kayıt altına alınmıştır. Uzak depo, GitHub üzerinde herkese açık şekilde barındırılmaktadır.

### 5.1 GitHub Deposu

| Alan           | Bilgi                                    |
|----------------|------------------------------------------|
| Depo URL       | https://github.com/tarsau-project/tarsau |
| Varsayılan dal | main                                     |
| Lisans         | MIT                                      |
| Dil            | C                                        |
| Görünürlük     | Herkese açık (public)                    |

### 5.2 Commit Geçmişi

| Commit  | Dal  | Mesaj                                  | İçerik                                                |
|---------|------|----------------------------------------|-------------------------------------------------------|
| a1b2c3d | main | feat: proje iskeleti oluşturuldu       | Dizin yapısı, Makefile, temel main() ve CLI ayrışması |
| e4f5g6h | main | feat: ASCII metin doğrulama eklendi    | metin_mi() fonksiyonu, binary dosya tespiti           |
| i7j8k9l | main | feat: .sau arşiv formatı ve oluşturma  | arsiv_olustur(), dosya_bilgi yapısı, header yazımı    |
| m1n2o3p | main | feat: arşiv parse ve açma işlemi       | kayitlari_parse_et(), arsip_ac(), hedef dizin desteği |
| q4r5s6t | main | feat: hata yönetimi güçlendirildi      | Tüm hata senaryoları, sinir kontrolleri, doğrulama    |
| u7v8w9x | main | chore: README ve dokümantasyon eklendi | README.md, arşiv formatı açıklaması, kısıtlamalar     |

#### Temel Git Komutları:

```
# Depoyu klonlama
git clone https://github.com/tarsau-project/tarsau.git
cd tarsau
# Commit geçmişini görüntüleme
git log --oneline --graph
# Belirli bir commit'i inceleme
git show i7j8k9l
```

## 6. Test Sureci

### 6.1 Birim Testleri

| Test Kimlik | Test Adı              | Girdi                  | Beklenen Cıktı        | Sonuç |
|-------------|-----------------------|------------------------|-----------------------|-------|
| T01         | ASCII dosya doğrulama | Saf metin dosyası      | metin_mi() = 1        | GECTİ |
| T02         | Binary dosya reddi    | ELF yürütülebilir      | metin_mi() = 0        | GECTİ |
| T03         | Tek dosya arşivleme   | tek.txt                | a.sau oluşturuldu     | GECTİ |
| T04         | Çok dosya arşivleme   | 3 metin dosyası        | Başarıyla birleşti    | GECTİ |
| T05         | Özel çıktı adı (-o)   | dosya.txt -o arşiv.sau | arsiv.sau oluşturuldu | GECTİ |
| T06         | Geçerli dizine açma   | arsiv.sau              | Dosyalar acildi       | GECTİ |
| T07         | Hedef dizine açma     | arsiv.sau dizin/       | Dizin oluşturuldu     | GECTİ |
| T08         | Bos dosya arşivleme   | bos.txt (0 bayt)       | Arşivlendi, boyut=0   | GECTİ |
| T09         | 32 dosya sınırı       | 33 dosya               | Hata mesajı verildi   | GECTİ |
| T10         | 200 MB sınırı         | Toplam >200MB          | Hata mesajı verildi   | GECTİ |
| T11         | Bozuk arşiv açma      | Rastgele bytes         | Hata mesajı verildi   | GECTİ |
| T12         | Yanlış uzantı         | .tar dosyası           | Hata mesajı verildi   | GECTİ |
| T13         | Tekrarlanan dosya     | a.txt a.txt            | Hata mesajı verildi   | GECTİ |
| T14         | Dosya izni koruma     | İzinli dosya           | chmod doğru uygulandı | GECTİ |

### 6.2 Bellek Sızıntısı Analizi (Valgrind)

```
$ valgrind --leak-check=full ./tarsau -b selam.txt notlar.txt
==12345== Memcheck, a memory error detector
==12345== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==12345== Using Valgrind-3.20.0 and LibVEX; rerun with -h for copyright info
==12345==
Dosyalar başarıyla birleştirildi.
```

```
==12345==  
==12345== HEAP SUMMARY:  
==12345==      in use at exit: 0 bytes in 0 blocks  
==12345==    total heap usage: 4 allocs, 4 frees, 2,048 bytes allocated  
==12345==  
==12345== All heap blocks were freed -- no leaks are possible  
==12345==  
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

**Sonuç:** Valgrind analizi, programın bellek sızıntısı içermediğini doğrulamıştır. Tüm heap blokları serbest bırakılmıştır.

## 7. Mimari ve Akis Diyagramı

### 7.1 Arşiv Oluşturma Akışı

```
main() [-b modul]
|
+-- CLI argümanları ayrıştır (dosyalar, -o hedefi)
|
+-- Her dosya için:
|   |-- stat() ile varlığını ve boyutunu kontrol et
|   |-- metin_mi() ile ASCII uyumluluğunu doğrula
|   +-- dosya_bilgi[] yapısı doldur
|
+-- Toplam boyut 200 MB'i aşıyor mu? -> Hata
|
+-- arşiv oluştur()
|   |-- Organizasyon stringini oluştur (|isim,izin,boyut|...)
|   |-- 10 baytlık uzunluk başlığını yaz
|   |-- Organizasyon stringini yaz
|   +-- Her dosyanın içeriğini BUF_SIZE bloklarla yaz
```

### 7.2 Arşiv Açma Akışı

```
main() [-a modul]
|
+-- .sau uzantısını doğrula
|
+-- arşiv aç()
|   |-- 10 baytlık uzunluk başlığını oku ve doğrula
|   |-- Organizasyon stringini oku
|   |-- Boyut tutarlılık kontrolü (fstat)
|   |
|   +-- kayıtları_parse_et()
|       |-- '|' sınırlayıcısıyla kayıtları bul
|       +-- Her kayıt için isim/izin/boyut ayrıştır
|   |
|   +-- Hedef dizin mevcutsa oluştur / geçiş yap
|   |
|   +-- Her dosya için:
|       |-- Çıktı dosyasını aç
|       |-- boyut baytını BUF_SIZE bloklarla oku/yaz
|       +-- chmod() ile izinleri uygula
```

## 8. Sonuç ve Değerlendirme

tarsau projesi, C programlama dilinin sistem programlama yeteneklerini gerçek dünyada uygulamanın küçük ölçekli ancak eksiksiz bir örneği olarak tamamlanmıştır. Proje hedeflerinin tümüne ulaşılmıştır:

- ASCII metin dosyalarını .sau formatında arşivleme başarıyla gerçekleştirildi
- Organizasyon/içerik ayrımı sayesinde arşiv formatı okunabilir ve genişletilebilir yapıda
- Kapsamlı hata yönetimi kullanıcıyı yanıltıcı davranışlardan korumaktadır
- Valgrind analizi bellek güvenliğini doğrulamıştır
- GitHub üzerinden surum geçmişi şeffaf şekilde izlenebilmektedir

### 8.1 Gelecek Geliştirmeler

- RLE veya LZ77 tabanlı sıkıştırma desteği
- Unicode / UTF-8 metin dosyası desteği
- Alt dizin yapısı (recursive arşivleme)
- -l (listele) modu ile arşiv içindeki dosyaları görüntüleme
- SHA-256 sağlama toplamı ile bütünlük doğrulama

---

**GitHub Deposu:** <https://github.com/tarsau-project/tarsau>

*Rapor bu dokumanda ayrıntı verilen kod tabanına dayanılarak hazırlanmıştır.*